

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Assessment Tool Weightage: 20%

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 25

Code of Conduct

I **K.MAHESH BABU(192525122)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **K.MAHESH BABU(192525122)**

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis

Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

Question 1

A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.

Answer

Identification of Error

The technical issue is **Data Skewness** (specifically **Partition Skew**). While the input splits are uneven, the primary failure mechanism is the imbalance in data distribution across reducers. This leads to the "Long Tail" problem, where a few reducers are overwhelmed with massive keys while others remain idle, eventually causing the job to time out or fail due to **Out-of-Memory (OOM)** errors on the overloaded nodes.

Cause Analysis

- **Poor Partitioning Logic:** The default `HashPartitioner` may be mapping a disproportionately large number of records to a single reducer because of a high frequency of specific keys (e.g., null values or common categories).
- **Input Split Misconfiguration:** If the input files are non-splittable (like compressed GZIP files) or the `mapreduce.input.fileinputformat.split.maxsize` is poorly tuned, the initial Map phase creates uneven workloads that propagate to the Shuffle and Reduce phases.
- **Resource Starvation:** Idle reducers hold onto container slots while waiting for the skewed reducer to finish, leading to cluster-wide resource underutilization and potential job preemption by the YARN scheduler.

Corrective Actions

- **Implement Salting:** Add a random prefix or suffix (a "salt") to the keys before the shuffle phase. This breaks down large keys into sub-keys, ensuring they are distributed across multiple reducers. The salt is removed during the final aggregation.
- **Custom Partitioner:** Develop a domain-specific partitioner that uses a **Total Order Partitioner** or a sampling-based approach to ensure a more uniform distribution of keys based on the actual data distribution.
- **Combiner Functions:** Use a `Reducer` as a `Combiner` (where the operation is associative and commutative) to perform local aggregation on the Mapper side, significantly reducing the volume of data transferred over the network to the reducers.
- **Adjust Split Size:** Ensure input data uses splittable compression formats (like BZip2 or LZO) and tune `mapred.min.split.size` to ensure Mappers receive balanced workloads from the start.

Question 2

An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.

Answer

Analysis of Data Unavailability in HDFS

Identification of Error

The primary technical issue is a **violation of the HDFS Fault Tolerance principle**, specifically regarding the **Replication Factor (RF)** or **Rack Awareness** configuration. In a standard HDFS deployment, the loss of a single DataNode should not result in data unavailability unless the blocks stored on that node have no viable replicas elsewhere in the cluster. This suggests a configuration where the replication factor is set to 1 ($RF=1$), or a logical failure in the NameNode's block placement policy.

Cause Analysis

- **Insufficient Replication Factor:** If `dfs.replication` is configured to 1, each block exists only on a single DataNode. Consequently, if that node fails, the NameNode cannot redirect client requests to an alternate location, leading to immediate data loss/unavailability.
- **NameNode Metadata Corruption or Staleness:** If the NameNode fails to update its **EditLogs** or **FsImage** correctly, it may point to the failed node for blocks that have theoretically been replicated but are not physically accessible, causing a mismatch between the metadata and the physical block pool.
- **Lack of Rack Awareness:** If all replicas of a block were inadvertently placed on the same rack (or the same node due to a misconfigured `BlockPlacementPolicy`), a single point of failure (top-of-rack switch or node power) renders the data inaccessible despite a higher theoretical RF.

Corrective Action

- **Adjust Replication Factor:** Modify the `hdfs-site.xml` configuration to set `dfs.replication` to the industry standard of 3. This ensures that data is stored on three separate nodes across at least two different racks.
- **Implement Rack Awareness:** Configure the `net.topology.script.file.name` to ensure the NameNode understands the physical topology, preventing it from placing multiple replicas on the same failure domain.
- **HDFS Balancer and FSCK:** Run the `hdfs fsck /` command to identify under-replicated blocks and use the `balancer` utility to redistribute blocks across the cluster to ensure high availability and health.
- **High Availability (HA) NameNode:** Deploy a **Standby NameNode** using Quorum Journal Manager (QJM) to ensure that if the Active NameNode fails or loses connectivity to DataNodes, the cluster remains operational.

Question 3

A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.

Answer

Identification of Error

The primary technical issue is a **lack of idempotency** within the data pipeline. In Apache NiFi, this typically manifests as **duplicate delivery** where the same data packet (FlowFile) is processed or ingested multiple times into the destination analytics store (e.g., HDFS, Hive, or a NoSQL database) without a mechanism to detect or reject existing records.

Cause Analysis

- **NiFi Processor Retries:** If a downstream system experiences a timeout but successfully commits the data, NiFi may interpret the timeout as a failure and retry the transmission, leading to duplicates.
- **Lack of Atomic Transactions:** Many Big Data sinks do not support ACID transactions across distributed writes, meaning a partial failure followed by a restart can re-ingest previously written data.
- **Non-Unique Business Keys:** The analytics store may lack primary key constraints or unique indexes, allowing the storage engine to accept identical records.
- **Overlapping Fetch Logic:** In the ETL source phase, if NiFi uses a `QueryDatabaseTable` processor without a properly tracked 'Maximum Value Column', it may re-fetch the same rows during subsequent execution cycles.

Corrective Action

- **Implement Record Deduplication:** Use the `DetectDuplicate` processor in NiFi, leveraging a distributed cache (like Redis or HBase) to store and compare FlowFile attributes or content hashes.

- **Enable Idempotent Writes:** Configure the destination sink to perform **Upserts** (Update or Insert) rather than simple Appends. For example, using `'PutDatabaseRecord'` with a defined unique key.
- **State Management:** Ensure processors like `'ListHDFS'` or `'ListSFTP'` are correctly maintaining state so they only pick up new files created since the last execution.
- **Deterministic Record ID Generation:** Generate a UUID or hash based on the record content early in the NiFi flow and use this as the primary identifier in the analytics store to prevent collisions.

Hadoop / Integration Integration

In a Hadoop ecosystem, duplicates often occur because HDFS is an append-only file system. To resolve this at the integration layer, one should use **Apache Hive ACID tables** or **Apache Hudi/Iceberg**, which provide the necessary upsert capabilities to handle late-arriving or duplicate data packets from NiFi automatically.

Question 4

A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.

Answer

Identification of Error

The technical issue is **Resource Starvation** caused by a **FIFO (First-In-First-Out) Scheduling** bottleneck. In a default YARN configuration, the first job submitted consumes all available containers in the cluster. Subsequent jobs, regardless of their priority or size, are placed in a wait state until the preceding job releases resources. This leads to high latency for small jobs and poor cluster utilization when a single large application monopolizes the `'ResourceManager'` queue.

Cause Analysis

The root cause lies in the lack of **multi-tenancy resource isolation**.

- **Monolithic Allocation:** The FIFO scheduler does not support preemption or resource sharing; it treats the cluster as a single-user environment.
- **Lack of Max-Capacity Constraints:** Without defined limits on how much of the cluster a single user or queue can occupy, a single "heavy" MapReduce or Spark job can saturate all `'NodeManager'` vCores and memory.
- **Head-of-Line Blocking:** Small, interactive queries are blocked behind long-running batch processes, violating the requirements of a shared production environment.

Corrective Action

To resolve resource starvation, the cluster should transition to a more sophisticated YARN scheduler:

- **Implementation of Capacity Scheduler:** Divide the cluster into hierarchical queues (e.g., `'Production'`, `'Development'`, `'Ad-hoc'`). Assign a **Minimum Guaranteed Capacity** to each queue to ensure no user is completely starved, while allowing "elasticity" to use idle resources from other queues.
- **Fair Scheduler Configuration:** Enable the Fair Scheduler to assign resources such that all apps get, on average, an equal share of resources over time. This supports **Preemption**, where the `'ResourceManager'` can kill containers from over-served applications to satisfy the minimum requirements of starved applications.
- **Resource Limits:** Set `'yarn.scheduler.capacity.<queue-path>.maximum-capacity'` to prevent any single queue from consuming 100% of the cluster during peak contention.

Hadoop Integration Concepts

In the YARN architecture, the **ResourceManager (RM)** acts as the ultimate authority for resource arbitration. By utilizing the **Scheduler** component (pluggable via `'yarn-site.xml'`), the RM can move away from simple

serial execution to **Dominant Resource Fairness (DRF)**. DRF is critical in Hadoop environments where jobs are not just memory-bound but also CPU-bound, ensuring that the allocation considers multiple resource types (vCores and RAM) simultaneously to prevent starvation in heterogeneous workloads.

Question 5

A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Answer

Identification of Error

The root technical issue is a **Consistency-Availability-Partition (CAP) theorem trade-off failure**, specifically a **Stale Read** or **Version Mismatch** error. The system is likely employing an **Asynchronous Replication** model where the backup process does not guarantee that all replicas or backup nodes have reached a consistent state (Quorum) before the primary node acknowledges a write or before the backup snapshot is finalized.

Cause Analysis

The technical cause stems from the lack of **Strict Consistency** protocols during the synchronization phase. Potential drivers include:

- **Eventual Consistency Lag:** In distributed systems like HDFS or NoSQL databases, if the backup is triggered while data is still in the 'eventual consistency' window, the backup captures an incomplete state.
- **Lack of Write Quorum:** If the system requires W writes to be successful but the backup service reads from a node that was not part of the latest write set ($R + W \leq N$), it retrieves outdated data.
- **Checkpointing Latency:** In Hadoop, if the Secondary NameNode or Checkpoint Node fails to merge the 'EditLog' into the 'FsImage' frequently enough, the restored metadata will point to an older state of the filesystem.

Corrective Action

To resolve this design flaw, the following architectural changes are required:

- **Implement Synchronous Replication:** Transition to a synchronous or semi-synchronous replication mode where a write is only considered successful once it is committed to both the primary and the backup/standby nodes.
- **Enforce Quorum-Based Reads:** Configure the backup utility to perform **Quorum Reads**, ensuring it queries enough nodes to guarantee the retrieval of the most recent version (timestamp-based validation).
- **Atomic Snapshots:** Utilize hardware-level or filesystem-level atomic snapshots (e.g., HDFS Snapshots) that freeze the state of the NameNode metadata and DataNode blocks simultaneously to prevent partial updates during the backup window.

Hadoop / Integration Concepts

In a **Hadoop High Availability (HA)** cluster, this issue is mitigated by using **Quorum Journal Nodes (QJN)**. The Active NameNode writes edit logs to a majority of JournalNodes; the Standby NameNode reads these logs to stay synchronized. If the backup strategy bypasses the QJN or fails to account for the 'lastAppliedTxId', it will inevitably restore a stale image of the distributed storage.